

DEPTH CHARGE

Personal Implementation Guide

Ali Raza (25K-0827) — Member 3

Random numbers • Menu & screens • Falling depth charges • Score, lives & levels • Sound effects • High-score file

Your folder / files	src/ali/ — random, screens, charges, sound, score, highscore (.inc), stubs_ali.inc
Your working dates	Week 0 – Week 6: 5 Oct – 22 Nov 2026 (demo Mon 23 Nov)
GitHub repo	github.com/muhammadas20/Submarine-Defense-Game-with-Sonar-Detection
Group Leader	Muhammad Anas (25K-0899)
Member 2	Syed Alyan Hussain (25K-0648)
Member 3	Ali Raza (25K-0827)
Instructor / Demo	Muhammad Owais / Monday 23 November 2026

Read this first

- The code in this guide is a **worked example**. Type it yourself, change it if you want, and make sure you can explain **every line**. The viva is individual and covers the **whole** game, not only your part.
- Course rule: code taken from AI tools must be **disclosed** in the final report, e.g. "Reference examples from an AI-generated implementation guide were studied and adapted."
- There are three guides (one per member). They use the same files, names and dates, so they fit together.

1. Your Job in One Page

You build the **enemies and the rules** of the game: random numbers, the depth charges that fall from the top, the score, lives and levels, all sound effects, the saved high score, and the menu / instructions / game-over screens. Your files are short and simple, so you can finish each step in a few days.

In simple words

- **Anas** makes the game run (loop, keys, torpedoes, hits). **Alyan** draws everything.
- **You** decide *what happens*: where charges appear, how fast they fall, how many points, when the game ends.
- Your procedures are small "tools" that the others call — e.g. Anas calls `AddScore` when a torpedo hits.

1.1 Your steps

Step	Task	File (in src/ali/)	Week · dates
—	Study, fork, clone, build, practice Pull Request	—	W0 · 5 – 11 Oct
C1	Random number generator	random.inc	W1 · 12 – 14 Oct
C2	Menu, instructions and game-over screens	screens.inc	W1–W2 · 15 – 25 Oct
C3	Depth-charge spawner and falling	charges.inc	W3 · 26 – 29 Oct
C4	Sound effects	sound.inc	W3 · 30 Oct – 1 Nov
C5	Score and lives	score.inc	W4 · 2 – 8 Nov
C6	Levels (game gets faster)	score.inc	W5 · 9 – 11 Nov
C7	High-score file	highscore.inc	W5 · 12 – 15 Nov
—	Bug fixes only, testing, report section	docs/report/ali.md	W6 · 16 – 22 Nov

1.2 What you give to the others / what you use from them

These are the "promises" between members. If your procedure takes and returns exactly what is written here, the three parts will fit together on the first try.

You provide	Input (registers)	Result	Used by
<code>SeedRandom, Rand</code>	Rand: EAX = range	EAX = random 0 ... range-1	InitSystem, SpawnCharge
<code>ShowMenu</code>	—	Sets <code>gameState</code> = PLAYING or EXIT	main
<code>GameOverScreen</code>	—	Sound, save high score, show final score	GameLoop
<code>SpawnCharge, UpdateCharges</code>	—	Create / move charges	GameLoop
<code>AddScore</code>	EAX = points	score += EAX, level may change	CheckCollisions
<code>LoseLife</code>	—	lives – 1; at 0 → ST_GAMEOVER	CheckCollisions
<code>ApplyLevel</code>	—	Sets fall / spawn speed for the level	InitGame, AddScore
<code>Play...Sound (5 sounds)</code>	—	Plays a tone; all registers kept	Torpedo, collision, sonar code
<code>LoadHighScore</code>	—	highScore = number in the file (or 0)	main

You use	Owner	What it does for you
---------	-------	----------------------

You use	Owner	What it does for you
<code>gameState, subX</code>	Anas	Current state; LoseLife puts the sub back in the middle
<code>InitGame</code>	Anas	Resets score/lives/level and calls your <code>ApplyLevel</code>
Drawing of charges and HUD	Alyan	Alyan's <code>BuildFrame</code> draws your <code>Charges</code> and shows score/lives/level

2. Your Weekly Timeline (5 Oct – 23 Nov)

We have **7 weeks** until the demo on **Monday 23 November**: one preparation week (Week 0) and six working weeks. Every week runs **Monday → Sunday**. Your deadline is **Sunday 9:00 pm**: by then the week's tasks must be finished, tested and sent to GitHub as a **Pull Request** (section 5).

Week	By Sunday 9 pm you must finish...	Pull Request title
Week 0 Mon 5 – Sun 11 Oct	Study section 4. Install VS 2022 + Irvine in <code>C:\Irvine</code> . Fork, clone and build by Fri 9 Oct . Practice PR: fill the Week 0 line of <code>docs/progress/ali.md</code> .	Week 0 – Ali – practice PR
Week 1 Mon 12 – Sun 18 Oct	C1 <code>SeedRandom + Rand</code> . Start C2 : <code>PrintAt, WaitAnyKey, ShowMenu</code> .	Week 1 – Ali – C1 + menu
Week 2 Mon 19 – Sun 25 Oct	C2 finish: <code>ShowInstructions, GameOverScreen</code> → M1 (menu works, game starts).	Week 2 – Ali – C2
Week 3 Mon 26 Oct – Sun 1 Nov	C3 <code>SpawnCharge + UpdateCharges</code> , and C4 the five sound effects.	Week 3 – Ali – C3 C4
Week 4 Mon 2 – Sun 8 Nov	C5 <code>AddScore + LoseLife</code> → M2 .	Week 4 – Ali – C5
Week 5 Mon 9 – Sun 15 Nov	C6 levels (<code>ApplyLevel</code> + tables) and C7 high-score file.	Week 5 – Ali – C6 C7
Week 6 Mon 16 – Sun 22 Nov	By Wed 18 Nov : fixes — after that only bug fixes . Run your tests; write <code>docs/report/ali.md</code> by Sat 21 Nov .	Week 6 – Ali – fixes + report

2.1 The weekly rhythm (same for all three members)

When	What to do
Monday	Sync with Anas's repo (get last week's merged code) and start a new branch for the week.
Mon – Fri	Do the week's step(s). Commit + push every time a small piece works (at least every 2 days).
Saturday	Test your step(s) with the "Test it" boxes. Fix bugs.
Sunday before 9:00 pm	Write 3 lines in <code>docs/progress/ali.md</code> , push, and open the Pull Request .
Sunday 9:30 pm	15-minute group call: what is done, what is stuck, next week's steps.
Sunday night / Monday	Anas merges all Pull Requests and tells the group "Week N merged — sync now".

2.2 Group milestones

Milestone	Date	What must work (whole group)
M1	25 Oct	Program runs; menu appears; submarine moves on screen with no flicker; random numbers work

Milestone	Date	What must work (whole group)
M2	8 Nov	Core game: torpedoes, falling charges, hits, explosions, score, lives, status bar
M3	18 Nov	Everything: sonar + hidden mines, sounds, levels, pause, high-score file, game-over screen
M4	22 Nov	All tests passed on a lab PC; final report written; ready for the 23 Nov demo

Tips

- **Behind schedule?** Say it in the Sunday call — don't hide it. Push only code that **builds**: an unfinished procedure keeps its STUB line, so the game still runs.
- Your plan is a little **ahead** of the proposal's timeline on purpose — the extra days are your safety buffer.

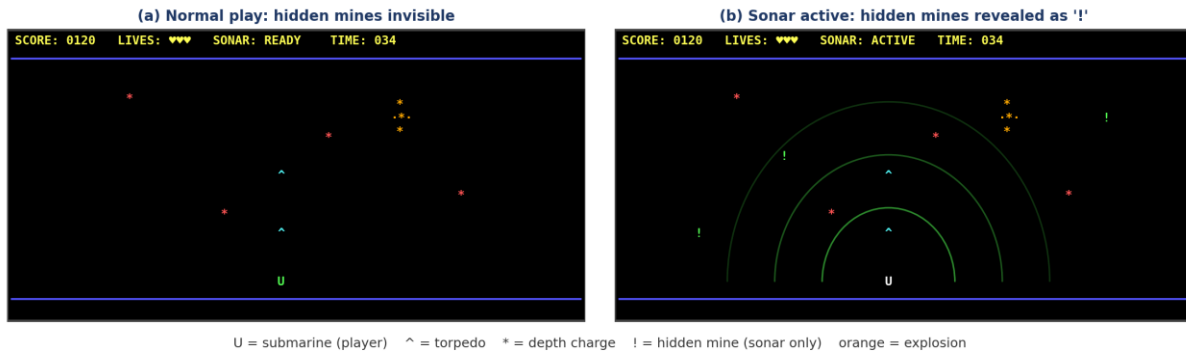
3. How the Game Works (5-minute version)

The whole game is **one loop** that repeats about **12 times per second** (every 80 ms). One repeat is called a **frame**. In every frame the program does the same five jobs, always in this order:

Job in every frame	What happens	Procedures	Who
1. Read keys	Check the keyboard without waiting ; move sub, fire, sonar, pause	<code>HandleInput</code>	Anas
2. Move things	Torpedoes go up, charges fall down, new charges appear, explosions age	<code>UpdateTorpedoes</code> <code>UpdateCharges</code> , <code>SpawnCharge</code> <code>UpdateExplosions</code>	Anas Ali (you) Alyan
3. Check hits	Torpedo hits charge → points; charge hits sub → lose a life	<code>CheckCollisions</code> → <code>AddScore</code> , <code>LoseLife</code>	Anas → Ali (you)
4. Draw	Draw everything into a memory copy of the screen, then show only what changed	<code>BuildFrame</code> , <code>DrawHUD</code> , <code>Render</code>	Alyan
5. Wait	Wait the rest of the 80 ms so the speed is the same on every PC	<code>FrameDelay</code>	Anas

Around the loop there are screens (menu, instructions, game over) and a **state** variable `gameState` that tells the loop what to do:

gameState	Meaning	What the loop does
ST_MENU (0)	Menu is showing	Loop stops, menu appears (ESC goes here)
ST_PLAYING (1)	Normal play	All five jobs
ST_SONAR (2)	Sonar is on (SPACE)	Nothing moves for 2 s; hidden mines shown as green 'I'
ST_PAUSED (3)	Paused (P)	Nothing moves; "PAUSED" text is drawn
ST_GAMEOVER (4)	No lives left	Loop stops, game-over screen appears
ST_EXIT (5)	Exit chosen in menu	Program ends



What the player sees: an 80×25 text screen (status bar on top, submarine 'U' at the bottom).

3.1 Project files and who owns them

There is **one** `.asm` file (`src/game.asm`). It **INCLUDEs** one `.inc` file per module. Every member has **one folder** and edits only that folder, so three people can work at the same time without conflicts. The full folder tree and the Git rules are in **section 5**.

Folder / file	What is inside	Owner
<code>src/game.asm</code>	<code>main</code> , <code>InitGame</code> , <code>GameLoop</code> , <code>FrameDelay</code> , all INCLUDE lines	Anas
<code>src/shared/constants.inc</code> , <code>data.inc</code>	Shared numbers and variables (everyone agrees first)	Anas (all agree)
<code>src/anas/</code>	<code>input.inc</code> , <code>torpedo.inc</code> , <code>collision.inc</code> , <code>stubs_anas.inc</code>	Anas
<code>src/alyan/</code>	<code>render.inc</code> , <code>hud.inc</code> , <code>explosion.inc</code> , <code>sonar.inc</code> , <code>stubs_alyan.inc</code>	Alyan
<code>src/ali/</code>	<code>random.inc</code> , <code>screens.inc</code> , <code>charges.inc</code> , <code>sound.inc</code> , <code>score.inc</code> , <code>highscore.inc</code> , <code>stubs_ali.inc</code>	Ali ← you
<code>docs/progress/<name>.md</code> <code>docs/report/<name>.md</code>	Your weekly log and your final-report section	Each member

3.2 The one object type everybody uses

The submarine is just a column number (`subX`). Every torpedo, charge and explosion is a 4-byte `GameObject` stored in an array. **active = 0** means the slot is free. **hidden = 1** means the charge is an invisible mine (it still moves and still hits — it is only *not drawn*).

```

data.inc (the comment shows who WRITES each variable; anyone may read it)
;=====
; data.inc - shared data (who WRITES each variable is in the comment)
;=====
GameObject STRUCT
    xPos    BYTE 0          ; column 0-79
    yPos    BYTE 0          ; row 0-24
    active   BYTE 0          ; 1 = slot in use
    hidden   BYTE 0          ; 1 = drawn only during sonar
GameObject ENDS

.data
gameState   BYTE  ST_MENU      ; Anas
subX        BYTE  40           ; Anas
score       DWORD 0           ; Ali
highScore   DWORD 0           ; Ali
lives       BYTE  3           ; Ali
level       BYTE  1           ; Ali
fireCooldown BYTE 0          ; Anas
lastFrame   DWORD 0           ; Anas
frameCount  DWORD 0           ; Anas
fallDelay   BYTE  6           ; Ali (set per level)
spawnDelay  BYTE  14          ; Ali (set per level)
fallTimer   BYTE  6           ; Ali
spawnTimer  BYTE  14          ; Ali
sonarStart  DWORD 0           ; Alyan
sonarReadyAt DWORD 0          ; Alyan
sonarPlayed BYTE 0           ; Alyan

Torpedoes    GameObject MAX_TORP DUP(<>) ; Anas
Charges      GameObject MAX_CHARGE DUP(<>) ; Ali
Explosions   GameObject MAX_EXPL  DUP(<>) ; Alyan
ExplTimer    BYTE  MAX_EXPL  DUP(0)      ; Alyan

ScreenBuf    WORD  SCREEN_W*SCREEN_H DUP(0720h) ; Alyan
PrevBuf      WORD  SCREEN_W*SCREEN_H DUP(0FFFFh) ; Alyan

```

4. What You Need to Know First

Study these in **Week 0 (5 – 11 Oct)** and keep going in Week 1 (Irvine, *Assembly Language for x86 Processors*, 8th ed.).

Topic	Simple explanation	Irvine	Practice
Basics, MOV , ADD , INC/DEC , CMP + unsigned jumps	Same basics as everyone	Ch 3–6	Count down from 10 and print each number
Procedures, USES , PUSH/POP	Your procedures are called by others, so they must not change their registers	Ch 5	A procedure that returns the bigger of EAX and EBX
Arrays of STRUCT	Walk Charges with ESI ; add esi , TYPE GameObject to go to the next one	Ch 4, 10	Array of 5 structs; set all active to 1
MUL and DIV	MUL result is in EDX:EAX . DIV gives EAX = quotient, EDX = remainder. Always `xor edx, edx` before `DIV`	Ch 7	Print 1234 / 200 and 1234 mod 200
Random numbers (LCG)	new seed = seed × a + c. Then (number mod range) gives 0 ... range–1	Ch 7	Print 20 random numbers between 0 and 9
Lookup tables	A list of numbers; use the level as the index: FallDelayTable[ebx]	Ch 4	Print the day name for 0–6

Topic	Simple explanation	Irvine	Practice
MACRO, PUSHAD/POPAD, INVOKE Beep	A macro is a short-cut that is copied in. <code>Beep(freq, ms)</code> plays a tone	Ch 10, 11	Play 5 notes going up
File procedures	<code>CreateOutputFile, OpenInputFile, WriteToFile, ReadFromFile, CloseFile</code>	Ch 11	Save a number to a file, run again and read it back
Screen text: <code>Clrscr, Gotoxy, SetTextColor, WriteString, WriteDec, ReadKey</code>	Simple screens drawn directly on the console	Ch 5	A menu that prints "1 / 2 / 3" and reads the choice

5. GitHub: Folder Structure and How to Push Your Work

5.1 The folder structure (already created by Anas)

Repository: <https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection>. Every member has **one folder** in `src/`. Because nobody edits another person's files, Git can always merge the three parts **without conflicts**.

```

folder structure
Submarine-Defense-Game-with-Sonar-Detection/
├── README.md      .gitignore  .gitattributes  .github/CODEOWNERS      Anas
├── Project.sln    Project.vcxproj  Project.vcxproj.filters  Anas (others NEVER commit)
├── src/
│   ├── game.asm          Anas    the only file that is assembled
│   ├── shared/
│   │   ├── constants.inc  Anas    (all three agree first)
│   │   └── data.inc       Anas    (all three agree first)
│   ├── anas/
│   │   ├── input.inc      torpedo.inc  collision.inc
│   │   └── stubs_anas.inc
│   ├── alyan/
│   │   ├── render.inc    hud.inc      explosion.inc  sonar.inc
│   │   └── stubs_alyan.inc
│   ├── ali/
│   │   ├── random.inc    screens.inc  charges.inc
│   │   ├── sound.inc     score.inc     highscore.inc
│   │   └── stubs_ali.inc
│   └── docs/
│       ├── proposal/    guides/      Anas
│       ├── progress/    anas.md      alyan.md      ali.md        each member: own weekly log
│       └── report/      anas.md      alyan.md      ali.md        each member: own report section

```

	Files
You may change	<code>src/ali/</code> (all files inside), <code>docs/progress/ali.md</code> , <code>docs/report/ali.md</code>
Never change	<code>src/game.asm</code> , <code>src/shared/</code> , <code>src/anas/</code> , <code>src/alyan/</code> , <code>Project.*</code> , <code>README.md</code> , other members' docs

Rules that prevent conflicts

- Never use `git add .` — add only your own paths (shown in the commands below).
- Your module's private variables and tables go in a `.data` section **inside your own .inc file** (e.g. `FallDelayTable` lives in `score.inc`). Only truly shared variables are in `data.inc`.
- Need a new shared constant/variable, or a change in `game.asm`? Open a GitHub **Issue** or message Anas. He adds it.
- Never add files to the Visual Studio project — it changes `Project.vcxproj`. All files already exist.

- Testing something in `game.asm` on your PC is fine — run `git restore src/game.asm` before you commit.
- Everyone installs Irvine in `C:\Irvine`, so the same project settings work on all three laptops.

5.2 One-time setup (Week 0 — finish by Fri 9 Oct)

1. Install **Visual Studio 2022** (*Desktop development with C++*), Irvine32 in `C:\Irvine`, and **Git for Windows** (`git-scm.com`). Use the *Git Bash* window for the commands below.
2. Accept the **collaborator invitation** e-mail from GitHub (from `muhammadanas20`).
3. Open <https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection> and click **Fork** → **Create fork**. You now have your own copy on GitHub.
4. Run these commands (replace YOUR-USERNAME with your GitHub username):

```
Git Bash — one time
git config --global user.name "Ali Raza"
git config --global user.email "the-email-of-your-github-account"

cd ~/source/repos
git clone https://github.com/YOUR-USERNAME/Submarine-Defense-Game-with-Sonar-Detection.git
cd Submarine-Defense-Game-with-Sonar-Detection

# connect Anas's repo as "upstream" (only once)
git remote add upstream https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection.git
git remote -v      # origin = your fork, upstream = Anas's repo
```

5. Open `Project.sln` in Visual Studio. Build with **Ctrl+Shift+B**, run with **Ctrl+F5**. It must print *"Depth Charge skeleton builds and runs OK"*.
6. **Practice Pull Request:** do the weekly steps below once with only `docs/progress/ali.md` (write your Week 0 line). When Anas merges it, you know the whole process works.

5.3 Every week, step by step

```
Git Bash — weekly

# ----- MONDAY: get last week's merged work, start this week's branch
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
git checkout -b week1-ali      # week2-ali, week3-ali, ...

# ----- EVERY TIME SOMETHING SMALL WORKS
git status                    # must list ONLY src/ali/... and docs/.../ali.md
git add src/ali docs/progress/ali.md docs/report/ali.md
git commit -m "C1: short description of what now works"
git push -u origin week1-ali   # first push of the week (later: just git push)

# ----- MID-WEEK (only if you need code Anas merged after Monday)
git fetch upstream
git merge upstream/main
```

Good commit messages start with the step number, e.g. `C2: Render skips unchanged cells` — then everyone can see what each commit did.

5.4 Sunday: open the Pull Request (before 9:00 pm)

1. Write your 3 lines in `docs/progress/ali.md`, commit and push (commands above).

2. Open **your fork** on GitHub. Click the yellow **Compare & pull request** button (or: *Pull requests* → *New pull request*).
3. Check the arrows: **base repository: muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection, base: main** ← **head repository: YOUR-USERNAME/Submarine-Defense-Game-with-Sonar-Detection, compare: week1-ali**.
4. Title: the one from your timeline, e.g. "*Week 1 – Ali – C1 + menu*". Description: what works, how you tested it, any problems.
5. Open the **Files changed** tab: only `src/ali/` and your own docs files may appear. If something else appears, fix it first (table below).
6. Click **Create pull request**. Anas merges it on Sunday night; on Monday you sync again (5.3).

5.5 If something goes wrong

Problem	Fix
<code>git status</code> lists <code>src/game.asm</code> (you tested something)	<code>git restore src/game.asm</code>
<code>Project.vcxproj / .sln / .filters</code> changed	<code>git restore Project.sln Project.vcxproj Project.vcxproj.filters</code>
You already committed a file outside your folder	<code>git checkout upstream/main -- <file></code> then commit "Undo change" and push
Conflict while merging <code>upstream/main</code>	Can only happen in your own files. Open the file, keep the right lines, delete the <<<<<<, =====, >>>>>> lines, then <code>git add <file></code> and <code>git commit</code> . Unsure? Ask Anas before pushing.
Push rejected ("fetch first")	<code>git pull origin <your-branch></code> then <code>git push</code>
Build error A2005 <i>symbol redefinition</i>	You wrote a procedure but its STUB line is still there — delete the STUB line.
Build error A2006 <i>undefined symbol</i>	The procedure has no real version and no STUB line — tell its owner / Anas.
Which branch am I on?	<code>git branch</code> (the one with *)

5.6 Your stub file

`src/ali/stubs_ali.inc` lets the game build before your code exists. When you finish a procedure, delete its **STUB** line **in the same commit**.

```
src/ali/stubs_ali.inc (starting version)

; stubs_ali.inc - EMPTY versions of Ali's procedures.
; Ali deletes ONE line as soon as the real procedure is written.
STUB MACRO procName          ; makes:  procName PROC / ret / procName ENDP
procName PROC
    ret
procName ENDP
ENDM

.code
STUB SeedRandom              ; called by InitSystem
STUB ShowMenu                ; called by main
STUB GameOverScreen          ; called by GameLoop
STUB LoadHighScore           ; called by main
STUB SpawnCharge              ; called by GameLoop
STUB UpdateCharges            ; called by GameLoop
STUB AddScore                 ; called by CheckCollisions
STUB LoseLife                 ; called by CheckCollisions
STUB ApplyLevel               ; called by InitGame
STUB PlayFireSound            ; called by FireTorpedo
STUB PlayExplosionSound        ; called by CheckCollisions
STUB PlaySonarSweep           ; called by UpdateSonar
STUB PlayLifeLostSound        ; called by LoseLife
STUB PlayGameOverSound        ; called by GameOverScreen
STUB SaveHighScore            ; called by GameOverScreen
```

Each **STUB** line creates an empty procedure that just returns. (**STUB** is a small **macro** — a good example of Irvine Ch 10.)

6. Your Steps

Do the steps in order. Each step says **which file**, **what it does in simple words**, the **logic**, an example, how to **test** it and the usual **mistakes**.

C1 Random number generator (Week 1 · 12 – 14 Oct)

File	src/ali/random.inc
You write	SeedRandom, Rand
You call	GetMseconds
Called by	Alyan's InitSystem, your SpawnCharge

In simple words

- A computer can't really be random, so we use a formula that *looks* random: $\text{seed} = \text{seed} \times 1103515245 + 12345$.
- We start the seed from the clock, so every game is different.
- Rand with EAX = 78 gives a number from 0 to 77 (a column).

The logic, step by step

1. SeedRandom: GetMseconds → store in seed.
2. Rand: save the range in EBX; $\text{seed} = \text{seed} \times a + c$ (MUL, then ADD); store the new seed.
3. shr eax, 16 (the high bits are more random), xor edx, edx, div ebx → the remainder in EDX is the answer.
4. Return it in EAX.

```
random.inc
SeedRandom PROC USES eax
    call GetMseconds           ; different seed every run
    mov  seed, eax
    ret
SeedRandom ENDP

;-----
; Rand: Receives EAX = range (must be > 0)
; Returns EAX = random number 0 .. range-1
;-----
Rand PROC USES ebx ecx edx
    mov  ebx, eax               ; save range
    mov  eax, seed
    mov  ecx, 1103515245        ; LCG multiplier
    mul  ecx                   ; EDX:EAX = seed * a
    add  eax, 12345             ; + c (mod 2^32 automatically)
    mov  seed, eax
    shr  eax, 16                ; upper bits are more random
    xor  edx, edx
    div  ebx                   ; EDX = EAX mod range
    mov  eax, edx
    ret
Rand ENDP
```

Test it — the step is DONE when...

- Calling Rand with EAX = 78 twenty times gives numbers 0 ... 77 with no clear pattern.
- Two runs of the program give different numbers.

Common mistakes

- and `eax, 7Fh` gives 0 ... 127, not 0 ... 77 — use `DIV` for an exact range.
- Calling `Rand` with `EAX = 0` → divide-by-zero crash.
- Forgetting `xor edx, edx` before `DIV`.

C2 Menu, instructions and game-over screens (Weeks 1–2 · 15 – 25 Oct)

File	src/ali/screens.inc
You write	<code>PrintAt</code> , <code>WaitAnyKey</code> , <code>ShowMenu</code> , <code>ShowInstructions</code> , <code>GameOverScreen</code>
You call	<code>Clrscr</code> , <code>Gotoxy</code> , <code>SetTextColor</code> , <code>WriteString</code> , <code>WriteDec</code> , <code>ReadKey</code> , <code>ReadKeyFlush</code> ; later <code>SaveHighScore</code> and <code>PlayGameOverSound</code>
Called by	<code>main</code> , <code>GameLoop</code>

In simple words

- These screens are not part of the moving game, so they draw **straight to the console** (not through Alyan's buffer).
- The menu waits for 1 (start), 2 (instructions) or 3 / ESC (exit) and sets `gameState`.

The logic, step by step

1. `PrintAt`: helper — set colour (`EAX`), go to `DH/DL`, print the string in `ESI`.
2. `ShowMenu`: `ReadKeyFlush` (forget keys from the game), `Clrscr`, print title, 3 options and the high score.
3. Wait in a loop: `Delay 30 + ReadKey` (the Delay stops it from using 100% CPU). '1' → `ST_PLAYING`; '2' → `ShowInstructions` then redraw; '3' or ESC → `ST_EXIT`.
4. `GameOverScreen`: play the game-over sound, call `SaveHighScore`, show "GAME OVER" and the final score; if the score is the new record show "NEW HIGH SCORE!". Wait 700 ms, flush keys, wait for any key.
5. `SaveHighScore` and `PlayGameOverSound` stay as STUB lines in `stubs_ali.inc` until C7 / C4.

screens.inc – helpers

```

;-----
; PrintAt: DH = row, DL = col, ESI = string, EAX = colour
;-----
PrintAt PROC USES edx
    call SetTextColor
    call Gotoxy
    mov  edx, esi
    call WriteString
    ret
PrintAt ENDP

;-----
; WaitAnyKey: waits for a key without burning 100% CPU
;-----
WaitAnyKey PROC USES eax
L1: mov  eax, 30
    call Delay
    call ReadKey
    jz   L1
    ret
WaitAnyKey ENDP

```

screens.inc - ShowMenu

```

ShowMenu PROC USES eax edx esi
    call ReadKeyFlush          ; forget keys pressed during the game
Redraw:
    mov  eax, ATTR_TEXT
    call SetTextColor
    call Clrscr
    mov  eax, ATTR_HUD
    mov  dh, 6
    mov  dl, 28
    mov  esi, OFFSET strTitle
    call PrintAt
    mov  eax, ATTR_TEXT
    mov  dh, 10
    mov  dl, 31
    mov  esi, OFFSET strOpt1
    call PrintAt
    inc  dh
    mov  esi, OFFSET strOpt2
    call PrintAt
    inc  dh
    mov  esi, OFFSET strOpt3
    call PrintAt
    mov  dh, 15
    mov  esi, OFFSET strHigh
    call PrintAt
    mov  eax, highScore
    call WriteDec
WaitKey:
    mov  eax, 30
    call Delay
    call ReadKey
    jz   WaitKey
    cmp  al, '1'
    je   StartGame
    cmp  al, '2'
    je   DoInstr
    cmp  al, '3'
    je   Quit
    cmp  al, 1Bh
    je   Quit
    jmp  WaitKey
StartGame:
    mov  gameState, ST_PLAYING
    ret
DoInstr:
    call ShowInstructions
    jmp  Redraw
Quit:
    mov  gameState, ST_EXIT
    ret
ShowMenu ENDP

```

screens.inc - GameOverScreen

```

GameOverScreen PROC USES eax edx esi
    call PlayGameOverSound
    call SaveHighScore        ; Ali : updates highScore if beaten
    mov  eax, ATTR_TEXT
    call SetTextColor
    call Clrscr
    mov  eax, lightRed + (black * 16)
    mov  dh, 8
    mov  dl, 30
    mov  esi, OFFSET strOver
    call PrintAt
    mov  eax, ATTR_TEXT
    mov  dh, 11
    mov  dl, 32
    mov  esi, OFFSET strFinal
    call PrintAt
    mov  eax, score

```

```

    call WriteDec
    mov  eax, score
    cmp  eax, 0
    je   NoRecord
    cmp  eax, highScore      ; equal -> we just set the record
    jne  NoRecord
    mov  eax, ATTR_HUD
    mov  dh, 13
    mov  dl, 28
    mov  esi, OFFSET strNewHigh
    call PrintAt
NoRecord:
    mov  eax, ATTR_TEXT
    mov  dh, 16
    mov  dl, 22
    mov  esi, OFFSET strAnyKey
    call PrintAt
    mov  eax, 700            ; ignore keys still held from play
    call Delay
    call ReadKeyFlush
    call WaitAnyKey
    ret
GameOverScreen ENDP

```

`ShowInstructions` is the same pattern: `Clrscr`, five `PrintAt` lines, `WaitAnyKey`. The strings are at the top of `screens.inc` in the reference folder.

Test it — the step is DONE when...

- **Milestone M1 (25 Oct):** the menu appears, 2 shows instructions, 1 starts the game, 3 exits.
- ESC in the game returns to the menu.

Common mistakes

- Waiting with `ReadKey` in a loop without `Delay` — 100% CPU.
- Not flushing keys — a W still held from the game skips the game-over screen.

C3 Depth-charge spawner and falling (Week 3 · 26 – 29 Oct)

File	src/ali/charges.inc
You write	<code>SpawnCharge</code> , <code>UpdateCharges</code>
You call	<code>Rand</code>
Called by	GameLoop (every frame while playing)

In simple words

- Every few frames a new charge appears at a **random column** on the top row.
- 1 in 5 charges is a **hidden mine** (`hidden = 1`) — Alyan won't draw it, but it is still there.
- Every few frames all charges move down one row. At the bottom they disappear.

The logic, step by step

1. `SpawnCharge`: `dec spawnTimer`; if not 0 → return. Else reset `spawnTimer = spawnDelay`.
2. Find a free slot in `Charges` (`active = 0`). If none, return.
3. `Column = Rand(78) + 1` (columns 1 ... 78). `Row = TOP_ROW`. `active = 1`, `hidden = 0`.
4. `Rand(5)`: if the result is 0 (1 chance in 5 = 20%) set `hidden = 1`.
5. `UpdateCharges`: `dec fallTimer`; at 0 reset it to `fallDelay`, then `inc yPos` of every active charge. If `yPos` reaches `BOTTOM_ROW`, set `active = 0`.

```

charges.inc

SpawnCharge PROC USES eax ecx esi
    dec     spawnTimer
    jnz     Done           ; not time yet
    mov     al, spawnDelay
    mov     spawnTimer, al

    mov     esi, OFFSET Charges    ; find a free slot
    mov     ecx, MAX_CHARGE

FindSlot:
    cmp     (GameObject PTR [esi]).active, 0
    je      Found
    add     esi, TYPE GameObject
    loop    FindSlot
    jmp     Done           ; all 15 slots in use

Found:
    mov     eax, MAX_COL - MIN_COL + 1    ; 78 possible columns
    call    Rand             ; 0..77
    add     al, MIN_COL      ; 1..78
    mov     (GameObject PTR [esi]).xPos, al
    mov     (GameObject PTR [esi]).yPos, TOP_ROW
    mov     (GameObject PTR [esi]).active, 1
    mov     (GameObject PTR [esi]).hidden, 0
    mov     eax, 5
    call    Rand             ; 0..4
    cmp     eax, 0           ; 1 chance in 5 = 20 %
    jne     Done
    mov     (GameObject PTR [esi]).hidden, 1

Done:
    ret
SpawnCharge ENDP

UpdateCharges PROC USES eax ecx esi
    dec     fallTimer
    jnz     Done           ; charges move every fallDelay frames
    mov     al, fallDelay
    mov     fallTimer, al
    mov     esi, OFFSET Charges
    mov     ecx, MAX_CHARGE

L1: cmp     (GameObject PTR [esi]).active, 0
    je      NextC
    inc     (GameObject PTR [esi]).yPos
    cmp     (GameObject PTR [esi]).yPos, BOTTOM_ROW
    jb      NextC
    mov     (GameObject PTR [esi]).active, 0    ; reached the sea bed

NextC:
    add     esi, TYPE GameObject
    loop    L1

Done:
    ret
UpdateCharges ENDP

```

Test it — the step is DONE when...

- Charges appear at random columns and fall steadily.
- About 1 in 5 is invisible — check **Charges** in the Watch window (hidden = 1).

Common mistakes

- Columns 0 or 79 — those are outside the playfield. Add **MIN_COL** after **Rand**.
- Moving every frame instead of using **fallTimer** — the game becomes impossible.

C4 Sound effects (Week 3 · 30 Oct – 1 Nov)

File	src/ali/sound.inc
You write	mPlaySound macro + PlayFireSound, PlayExplosionSound, PlayLifeLostSound, PlayGameOverSound, PlaySonarSweep
You call	Windows Beep (frequency, milliseconds)
Called by	Torpedo (Anas), collision (Anas), sonar (Alyan), your own code

In simple words

- Beep plays a tone, but the program **waits** while it plays. So sounds during play must be very short (20–30 ms).
- Long sounds are only used when the game is frozen anyway (sonar, life lost, game over).

The logic, step by step

1. Beep `PROTO`, `dwFreq:DWORD`, `dwDuration:DWORD` is already declared in `game.asm` (Anas).
2. The `mPlaySound` macro wraps `INVOKE Beep` with `PUSHAD/POPAD`, because Windows functions change `EAX`, `ECX` and `EDX`.
3. Fire = high and short (1200 Hz, 20 ms); explosion = low (150 Hz, 30 ms).
4. Game over = 4 notes going **down**; sonar = 5 notes going **up** (a loop that changes the frequency in `EBX`).

```

sound.inc
;=====
; sound.inc                                Owner: Ali Raza
;=====
;-----
; mPlaySound: plays one tone. Beep changes EAX/ECX/EDX, so we
; save all registers with PUSHAD / POPAD.
;-----
mPlaySound MACRO freq, dur
    pushad
    INVOKE Beep, freq, dur
    popad
ENDM

.code
PlayFireSound PROC
    mPlaySound 1200, 20          ; high and short
    ret
PlayFireSound ENDP

PlayExplosionSound PROC
    mPlaySound 150, 30          ; low
    ret
PlayExplosionSound ENDP

PlayLifeLostSound PROC
    mPlaySound 300, 120
    mPlaySound 200, 180
    ret
PlayLifeLostSound ENDP

PlayGameOverSound PROC USES ebx ecx
    mov ebx, 600
    mov ecx, 4
l1: mPlaySound ebx, 150          ; 600, 500, 400, 300 Hz
    sub ebx, 100

```

```

    loop L1
    ret
PlayGameOverSound ENDP

PlaySonarSweep PROC USES ebx ecx
    mov ebx, 400                ; start frequency
    mov ecx, 5
L1: mPlaySound ebx, 50          ; 400, 500, 600, 700, 800 Hz
    add ebx, 100
    loop L1
    ret
PlaySonarSweep ENDP

```

Test it — the step is DONE when...

- Each event has its own sound; the game doesn't stutter when firing quickly.
- With muted speakers the game still works normally.

Common mistakes

- Long beeps during play (e.g. 300 ms per shot) — the game stutters.
- Not saving registers around `Beep` — Anas's loop counter is destroyed.

C5 Score and lives (Week 4 · 2 – 8 Nov)

File	src/ali/score.inc
You write	<code>AddScore</code> , <code>LoseLife</code>
You call	<code>ApplyLevel</code> (still a stub until C6), <code>PlayLifeLostSound</code> (C4)
Called by	Anas's <code>CheckCollisions</code>

In simple words

- Anas finds a hit and calls `AddScore` with `EAX = 10` (or 25 for a hidden mine).
- When a charge hits the sub, Anas calls `LoseLife`. After 3 lives the game is over.

The logic, step by step

1. `AddScore`: add score, `eax`. (In C6 you will add the level calculation here.)
2. `LoseLife`: play the life-lost sound, `dec lives`. If `lives = 0` → `gameState = ST_GAMEOVER`.
3. Otherwise put the sub back in the middle (`subX = 40`) so the player can recover.

```

score.inc - LoseLife
;-----
; LoseLife: called when a charge hits the submarine
;-----
LoseLife PROC USES eax
    call PlayLifeLostSound    ; Ali (sound.inc)
    dec lives
    jnz StillAlive
    mov gameState, ST_GAMEOVER
    ret
StillAlive:
    mov subX, 40              ; respawn in the middle
    ret
LoseLife ENDP

```

Test it — the step is DONE when...

- Each hit adds 10 (25 for a hidden mine) — the status bar shows it.
- After 3 hits the game-over screen appears.
- **Milestone M2 (8 Nov):** core game playable with the whole group.

C6 Levels — the game gets faster (Week 5 · 9 – 11 Nov)

File	src/ali/score.inc
You write	level part of <code>AddScore</code> , <code>ApplyLevel</code> , the two lookup tables
You call	—
Called by	<code>AddScore</code> , Anas's <code>InitGame</code>

In simple words

- Level = score / 200 + 1 (maximum 9). Every 200 points the level goes up.
- Two small tables say how many frames to wait before charges move and before a new one appears. Higher level = smaller numbers = faster.

The logic, step by step

1. In `AddScore`, after adding: `EAX = score`, `xor edx, edx`, `div` by 200, `inc eax`; if above `MAX_LEVEL` use `MAX_LEVEL`.
2. If the new level is different from `level`, store it and call `ApplyLevel`.
3. `ApplyLevel`: `index = level - 1`; `fallDelay = FallDelayTable[index]`, `spawnDelay = SpawnDelayTable[index]`.

```
score.inc - tables, AddScore, ApplyLevel
;=====
; score.inc - score, lives, levels      Owner: Ali Raza
;=====
.data
; lookup tables indexed by (level - 1): higher level = smaller delay
FallDelayTable BYTE 6, 5, 5, 4, 4, 3, 3, 2, 2
SpawnDelayTable BYTE 14, 12, 11, 10, 9, 8, 7, 6, 5
MAX_LEVEL = LENGTHOF FallDelayTable

.code
;-----
; AddScore: EAX = points. Recalculates level = score/200 + 1
;-----
AddScore PROC USES eax ecx edx
    add     score, eax
    mov     eax, score
    xor     edx, edx          ; EDX must be 0 before DIV
    mov     ecx, 200
    div     ecx              ; EAX = score / 200
    inc     eax
    cmp     eax, MAX_LEVEL
    jbe     @F
    mov     eax, MAX_LEVEL
@@: cmp     al, level
    je      Done
    mov     level, al
    call    ApplyLevel
Done:
    ret
AddScore ENDP
```

```

;-----
; ApplyLevel: copies the delays for the current level
;-----
ApplyLevel PROC USES eax ebx
    movzx ebx, level
    dec ebx ; table index 0..8
    mov al, FallDelayTable[ebx]
    mov fallDelay, al
    mov al, SpawnDelayTable[ebx]
    mov spawnDelay, al
    ret
ApplyLevel ENDP

```

Test it — the step is DONE when...

- At 200 points the level becomes 2 and charges clearly fall faster.
- The level never goes above 9.

Common mistakes

- `div` without `xor edx, edx` → *Integer overflow* crash.
- Using `level` as the index instead of `level - 1` — reads past the end of the table.

C7 High-score file (Week 5 · 12 – 15 Nov)

File	src/ali/highscore.inc
You write	<code>LoadHighScore</code> , <code>SaveHighScore</code>
You call	<code>OpenInputFile</code> , <code>CreateOutputFile</code> , <code>ReadFromFile</code> , <code>WriteToFile</code> , <code>CloseFile</code>
Called by	main (load), your <code>GameOverScreen</code> (save)

In simple words

- The best score is kept in a small file `highscore.dat` (just 4 bytes: one DWORD).
- Load it once when the program starts; save it at game over only if the record was beaten.

The logic, step by step

1. `LoadHighScore`: `EDX` = file name → `OpenInputFile`. If `EAX` = `INVALID_HANDLE_VALUE` (no file yet, first run) → `highScore` = 0.
2. Otherwise read 4 bytes straight into `highScore` (`EDX` = `OFFSET highScore`, `ECX` = 4), then `CloseFile`.
3. `SaveHighScore`: if `score` ≤ `highScore` → return. Else `highScore` = `score`, `CreateOutputFile`, write 4 bytes, `CloseFile`.

```

highscore.inc
;=====
; highscore.inc - file handling Owner: Ali Raza
;=====
.data
hsFileName BYTE "highscore.dat", 0
hsHandle   DWORD ?

.code
LoadHighScore PROC USES eax ecx edx
    mov edx, OFFSET hsFileName
    call OpenInputFile ; EAX = handle
    cmp eax, INVALID_HANDLE_VALUE

```

```

    je    NoFile                ; first run: no file yet
    mov   hsHandle, eax
    mov   edx, OFFSET highScore ; read straight into the DWORD
    mov   ecx, SIZEOF highScore ; 4 bytes
    call  ReadFromFile          ; EAX = handle (still)
    mov   eax, hsHandle
    call  CloseFile
    ret
NoFile:
    mov   highScore, 0
    ret
LoadHighScore ENDP

SaveHighScore PROC USES eax ecx edx
    mov   eax, score
    cmp   eax, highScore
    jbe   Done                ; not a new record
    mov   highScore, eax
    mov   edx, OFFSET hsFileName
    call  CreateOutputFile     ; creates / overwrites
    cmp   eax, INVALID_HANDLE_VALUE
    je    Done
    mov   hsHandle, eax
    mov   edx, OFFSET highScore
    mov   ecx, SIZEOF highScore
    call  WriteToFile          ; EAX = handle
    mov   eax, hsHandle
    call  CloseFile
Done:
    ret
SaveHighScore ENDP

```

Test it — the step is DONE when...

- First run: the menu shows high score 0 and nothing crashes.
- Beat the score, exit, run again: the new high score is shown.
- **Milestone M3 (18 Nov):** all features work together.

Common mistakes

- Not checking `INVALID_HANDLE_VALUE` — crash on the first run.
- Opening the file in Notepad shows strange symbols — that is normal, it is binary.

7. Viva Preparation

Questions about your part

1. How does `Rand` work? Why `shr eax, 16`? How do you get a number from 0 to 77?
2. Why must `EDX` be 0 before `DIV`? What happens if it isn't?
3. How is the level calculated, and how does it change the speed? (lookup table, index = level - 1)
4. How are 20% of the charges made hidden?
5. Why does `mPlaySound` use `PUSHAD/POPAD`? Why are in-game sounds so short?
6. How is the high score stored? What happens on the very first run when there is no file?
7. Why does the menu use `Delay` inside its waiting loop?

Questions about the other members' parts (you can be asked these too)

1. How does the game loop decide what to do each frame? — Anas

2. How does `CheckCollisions` detect a hit? — Anas
3. How does the renderer avoid flicker? — Alyan
4. How can a hidden mine be invisible but still hit the sub? — Alyan

8. Your Checklist

✓	Step	Target date
☐	Forked, cloned, built; practice PR merged	Fri 9 – Sun 11 Oct
☐	Studied the topics in section 4	Sun 11 Oct
☐	C1 <code>SeedRandom</code> + <code>Rand</code> ; <code>ShowMenu</code> started — PR sent	Sun 18 Oct
☐	C2 <code>ShowMenu</code> , <code>ShowInstructions</code> , <code>GameOverScreen</code> — PR sent (M1)	Sun 25 Oct
☐	C3 <code>SpawnCharge</code> + <code>UpdateCharges</code> , C4 five sounds — PR sent	Sun 1 Nov
☐	C5 <code>AddScore</code> + <code>LoseLife</code> — PR sent (M2)	Sun 8 Nov
☐	C6 levels, C7 <code>LoadHighScore</code> + <code>SaveHighScore</code> — PR sent	Sun 15 Nov
☐	All your STUB lines deleted; only bug fixes after this (M3)	Wed 18 Nov
☐	Your tests passed; docs/report/ali.md written	Sat 21 Nov

About the example code

All code comes from one reference version of the game (folder `guide/reference_code/`). It was checked with a MASM-compatible assembler (JWasm) and the Irvine32 include files — the full game, the day-one starter repo and the in-between versions with stub files all assemble with 0 errors — but it has **not** been run on Windows yet. Test every step on your own PC exactly as the "Test it" boxes say, and treat the code as a starting point that you must understand.